# Queue structure Using JAVA

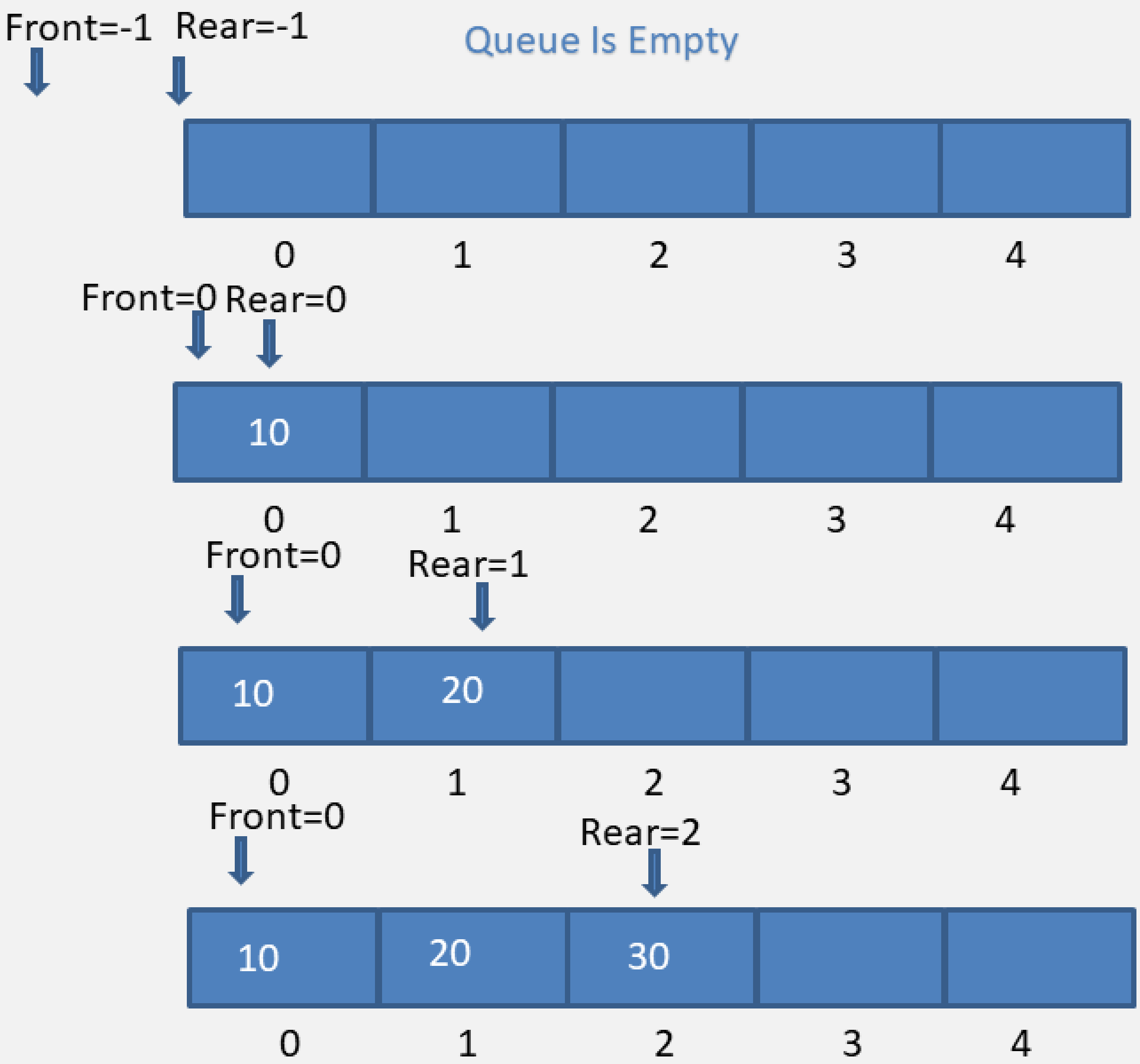
# Objective Of Todays Session….

1. concept Of Queue
2. Linear Queue Data structure
3. Dynamic Queue
4. Circular Queue
5. Double ended queue.

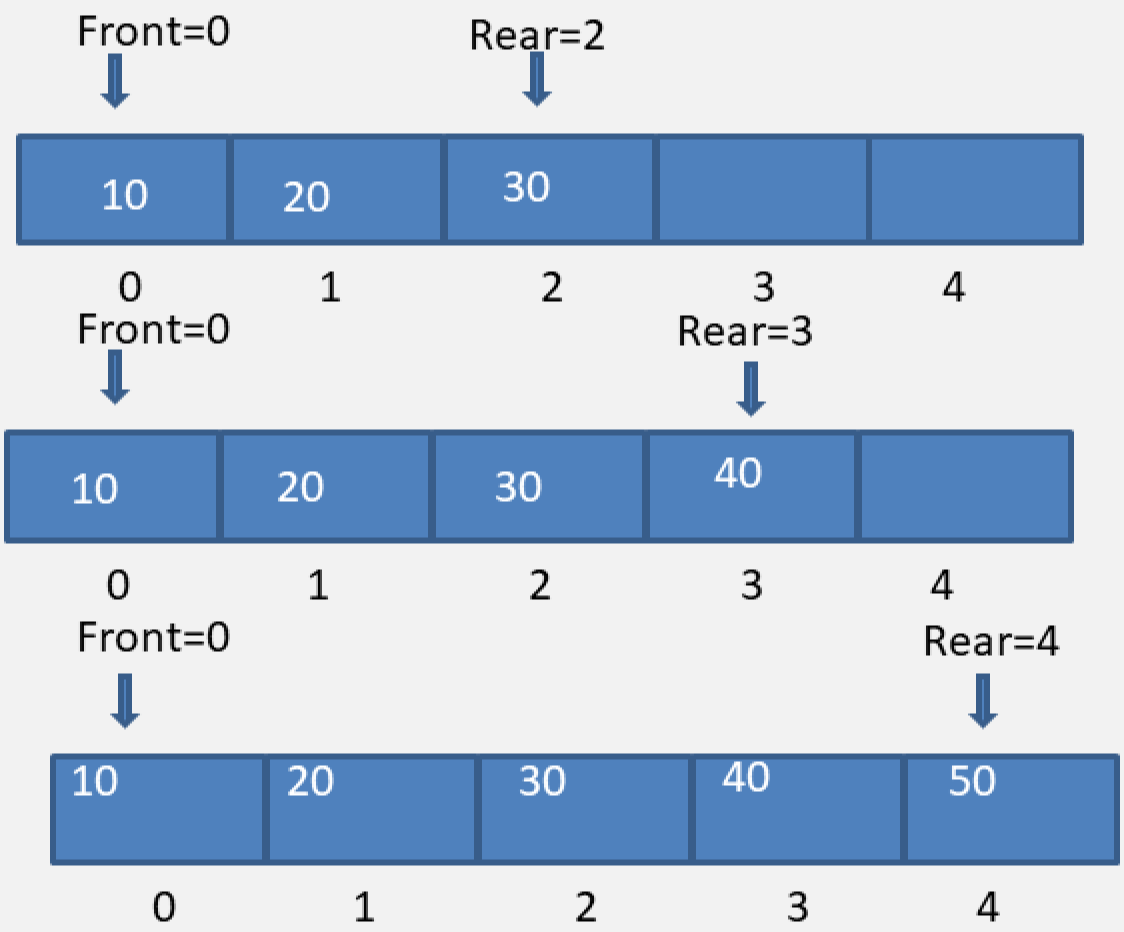
# Queue

A Queue is a linear structure which follows a particular order in which the operations are performed. The order is First In First Out (FIFO). A good example of a queue is any queue of consumers for a resource where the consumer that came first is served first. The difference between stacks and queues is in removing. In a stack we remove the item the most recently added; in a queue, we remove the item the least recently added.

# Queue Using Array (enQueue)

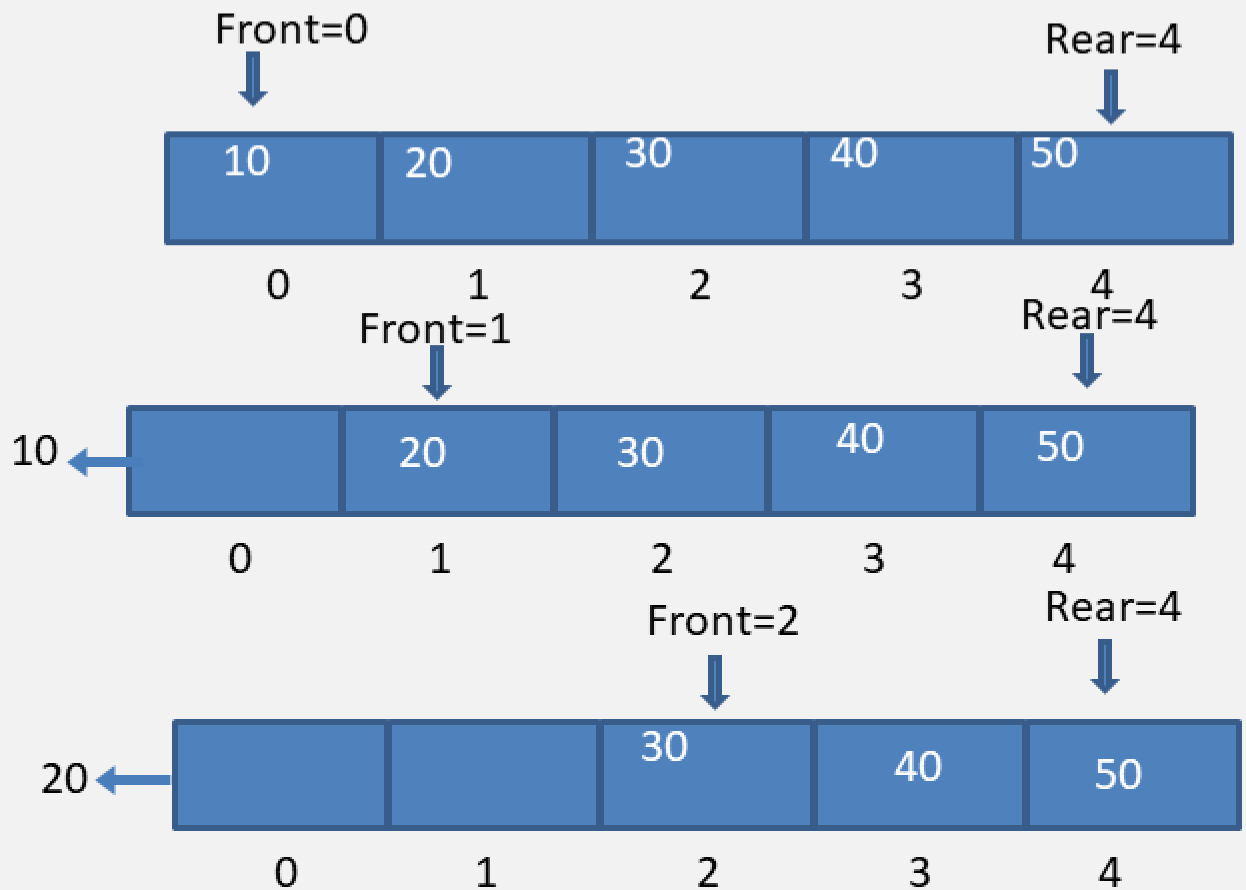


# Queue Using Array (enQueue)



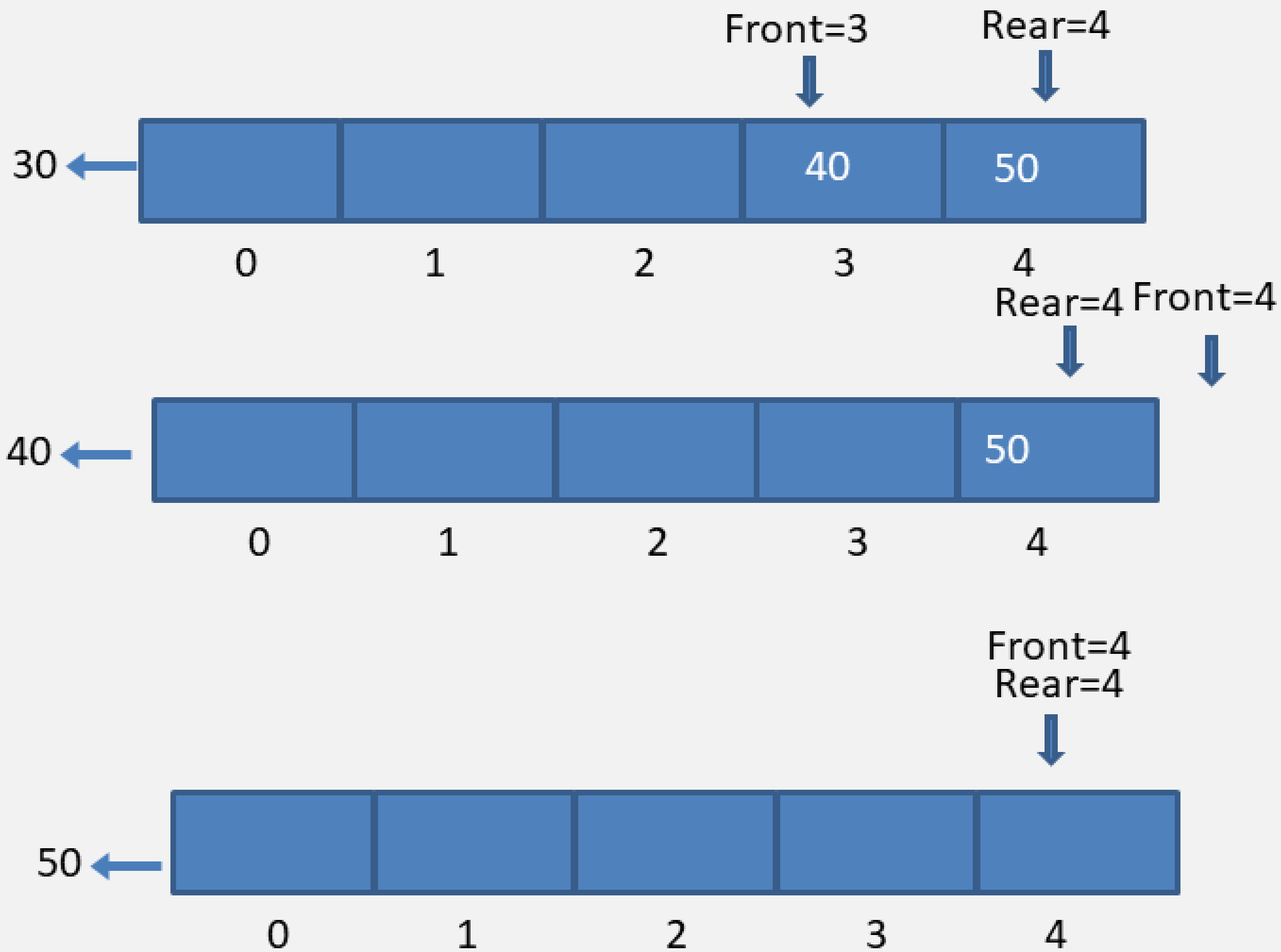

Queue Is Full

# Queue Using Array (deQueue)




Queue Is Full

# Queue Using Array (deQueue)




Queue Is Empty

# Double Ended Queue

Implementation of Double ended Queue

Here we will implement a double ended queue using a circular array. It will have the following methods:

push_back : inserts element at back

push_front : inserts element at front

pop_back : removes last element

pop_front : removes first element

get_back : returns last element

get_front : returns first element

empty : returns true if queue is empty

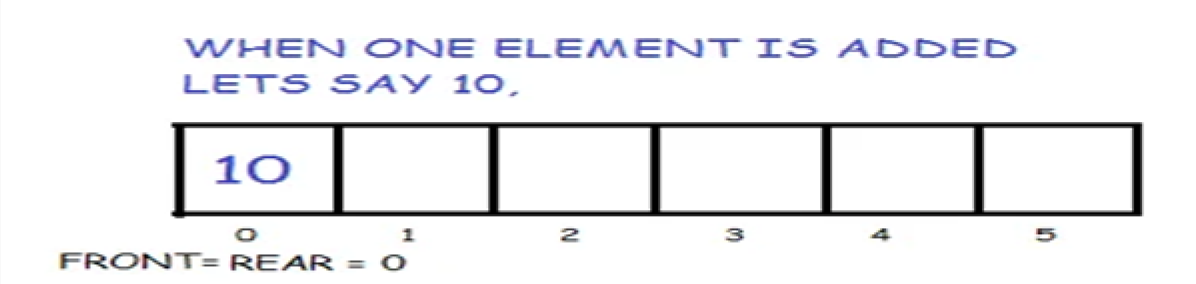
full : returns true if queue is full

# Double Ended Queue

Insert Elements at Front

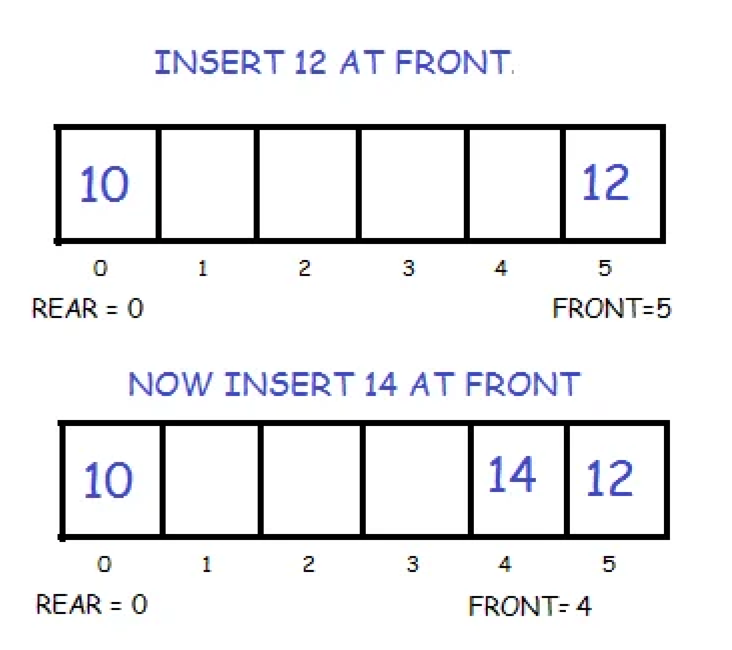
First we check if the queue is full. If its not full we insert an element at front end by following the given conditions :

If the queue is empty then intialize front and rear to 0. Both will point to the first element.



# Double Ended Queue

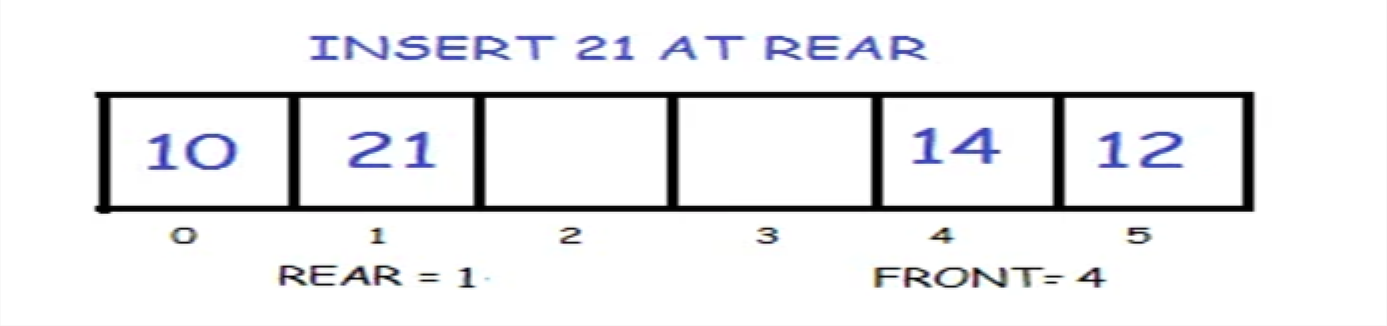
Else we decrement front and insert the element. Since we are using circular array, we have to keep in mind that if front is equal to 0 then instead of decreasing it by 1 we make it equal to SIZE-1.



# Double Ended Queue

Insert Elements at back
Again we check if the queue is full. If its not full we insert an element at back by following the given conditions:

If the queue is empty then intialize front and rear to 0. Both will point to the first element.
Else we increment rear and insert the element. Since we are using circular array, we have to keep in mind that if rear is equal to SIZE-1 then instead of increasing it by 1 we make it equal to 0.

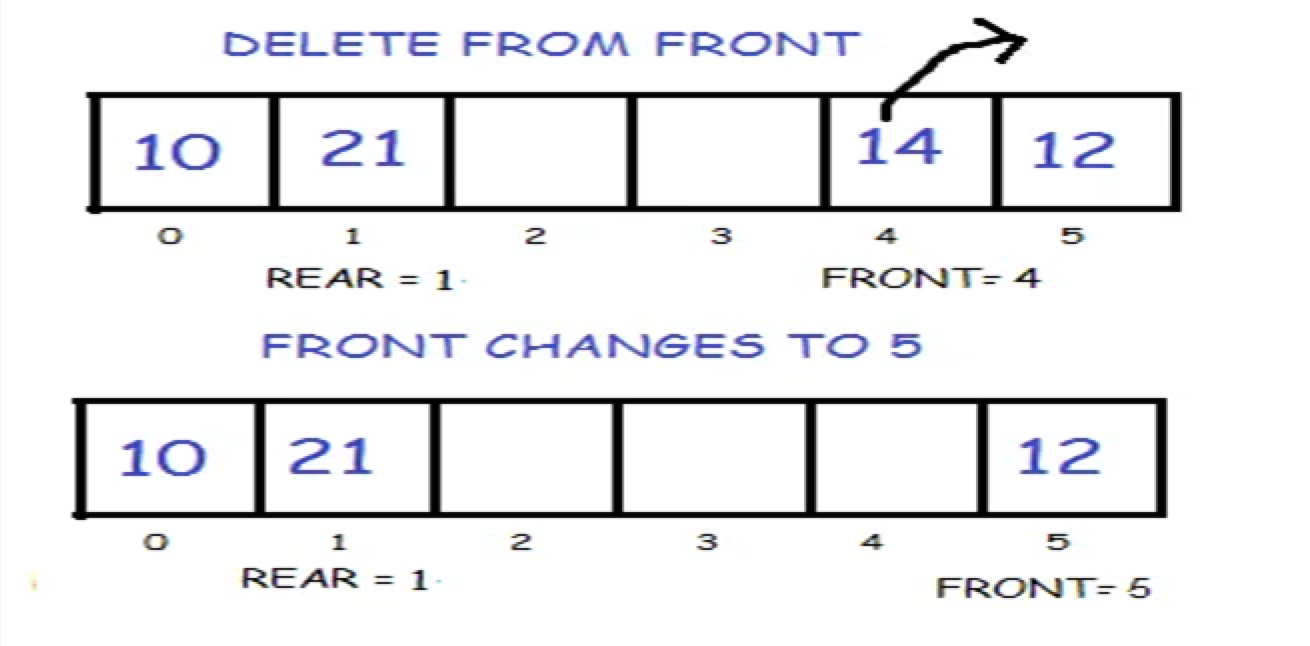


# Double Ended Queue

Delete First Element

In order to do this, we first check if the queue is empty. If its not then delete the front element by following the given conditions :

If only one element is present we once again make front and rear equal to -1.
Else we increment front. But we have to keep in mind that if front is equal to SIZE-1 then instead of increasing it by 1 we make it equal to 0.



# Double Ended Queue

Delete Last Element
Inorder to do this, we again first check if the queue is empty. If its not then we delete the last element by following the given conditions :

If only one element is present we make f

ront and rear equal to -1.
Else we decrement rear. But we have to keep in mind that if rear is equal to 0